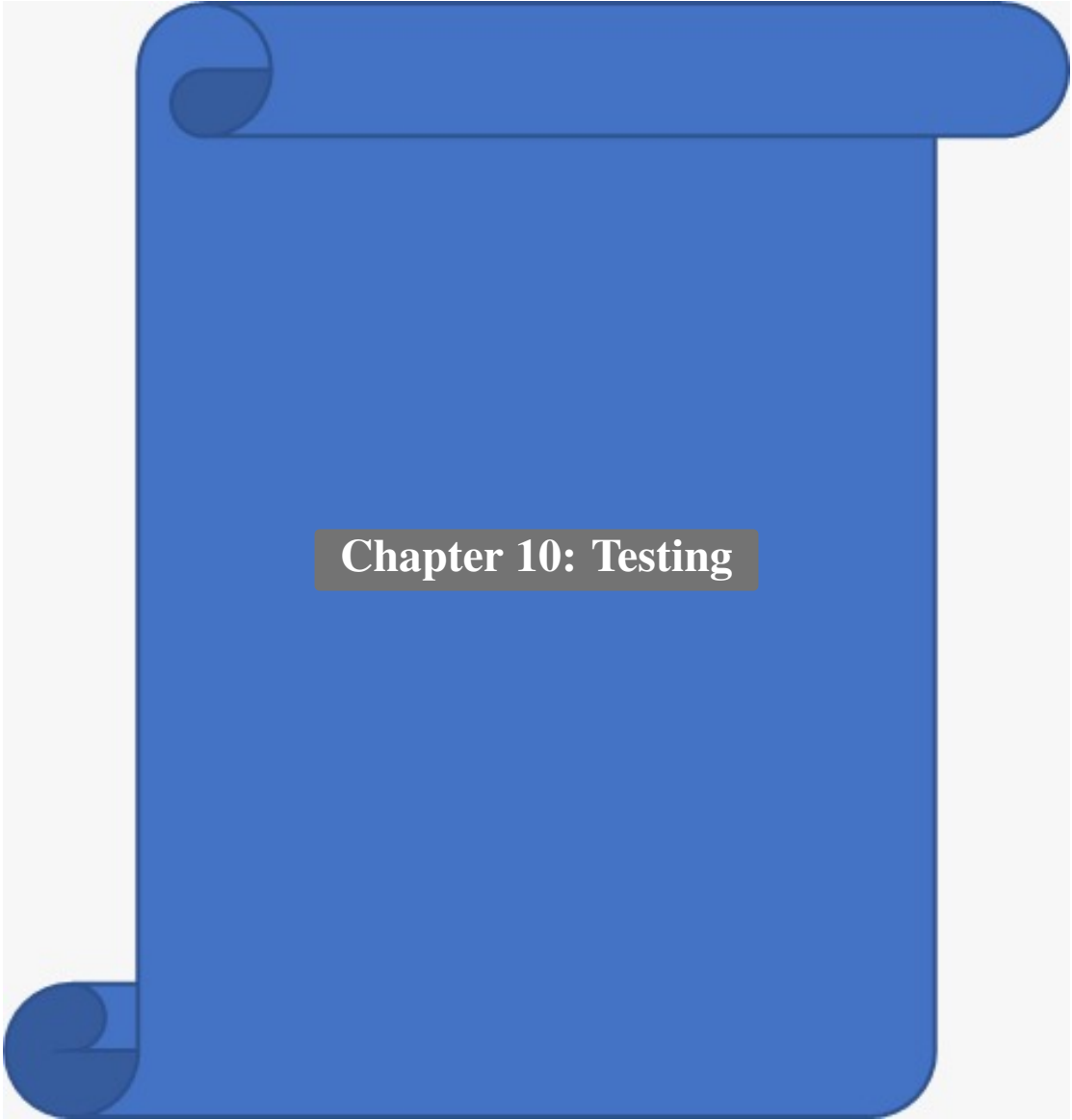




## Chapter 10: Testing

## 10.1 Testing Methodology

A comprehensive multi-layered testing approach was adopted to ensure the reliability, security, and usability of the AgriTrace platform. The testing process encompassed the following methodologies:

1. **Unit Testing:** Individual functions and components were tested in isolation to verify correct behavior. Firebase service functions (e.g., `createListing`, `claimListing`, `calculateCarbonOffset`) were tested with various input combinations.
2. **Integration Testing:** End-to-end workflows were tested to ensure proper interaction between modules. For example, the complete listing lifecycle (creation → claiming → pickup → delivery → recycling) was tested across all three user roles.
3. **Static Analysis:** ESLint with TypeScript plugin performed automated code quality checks. The `npm run lint` command was integrated into the development workflow to catch issues early.
4. **Type Checking:** TypeScript's strict mode (`npm run typecheck`) verified type safety across all 94 source files, catching potential runtime errors at compile time.
5. **Security Testing:** Firestore security rules were tested using the Firebase Emulator Suite. Unauthorized access attempts were simulated for all collections to ensure proper role-based access control.
6. **Cross-Browser Testing:** The application was tested on Chrome 120+, Firefox 121+, Safari 17+, and Edge 120+ to ensure rendering consistency and feature compatibility.
7. **Responsive Testing:** All pages were verified across desktop (1920×1080), tablet (768×1024), and mobile (375×667) viewports using Chrome DevTools device emulation.
8. **Performance Testing:** Page load times, Firestore query latency, and image upload speeds were benchmarked to ensure acceptable performance under typical usage conditions.

## 10.2 Test Cases

Comprehensive testing was performed across all modules of the AgriTrace platform. The following table presents key test cases:

| ID   | Module         | Test Case                  | Expected Result       | Actual Result           | Status |
|------|----------------|----------------------------|-----------------------|-------------------------|--------|
| TC01 | Authentication | Valid email/password login | Redirect to dashboard | Redirected to dashboard | Pass   |

| ID   | Module         | Test Case                 | Expected Result                              | Actual Result                | Status |
|------|----------------|---------------------------|----------------------------------------------|------------------------------|--------|
| TC02 | Authentication | Invalid credentials login | Show error message                           | Error displayed              | Pass   |
| TC03 | Authentication | New user registration     | Account created, role selection shown        | Account created successfully | Pass   |
| TC04 | Authentication | Password reset request    | Reset email sent                             | Email sent                   | Pass   |
| TC05 | Role Selection | Select Farmer role        | User profile updated, farmer dashboard shown | Role assigned correctly      | Pass   |
| TC06 | Role Selection | Select Agent role         | User profile updated, agent dashboard shown  | Role assigned correctly      | Pass   |
| TC07 | Farmer         | Create new listing        | Listing saved to Firestore with OPEN status  | Listing created              | Pass   |
| TC08 | Farmer         | Upload waste photos       | Photos stored in Firebase Storage            | Photos uploaded              | Pass   |
| TC09 | Farmer         | Cancel own listing        | Status changed to CANCELLED                  | Listing cancelled            | Pass   |
| TC10 | Agent          | View available tasks      | Open listings displayed                      | Listings shown               | Pass   |
| TC11 | Agent          | Claim a listing           | Status changes to ASSIGNED                   | Listing claimed              | Pass   |
| TC12 | Agent          | Start pickup              | Status changes to IN_TRANSIT                 | Status updated               | Pass   |
| TC13 | Agent          | Mark delivered            | Status changes to DELIVERED                  | Delivery confirmed           | Pass   |
| TC14 | Admin          | View all users            | All registered users listed                  | Users displayed              | Pass   |
| TC15 | Admin          | Edit user role            | Role updated in Firestore                    | Role changed                 | Pass   |
| TC16 | Admin          | Mark as recycled          | Status changes to RECYCLED                   | Status updated               | Pass   |
| TC17 | Admin          | View analytics            | Charts and stats rendered                    | Analytics displayed          | Pass   |

| ID   | Module       | Test Case                     | Expected Result                                   | Actual Result           | Status |
|------|--------------|-------------------------------|---------------------------------------------------|-------------------------|--------|
| TC18 | Payment      | Razorpay checkout             | Payment processed, status updated                 | Payment successful      | Pass   |
| TC19 | Carbon       | Calculate carbon offset       | Correct CO <sub>2</sub> value based on waste type | Values match factors    | Pass   |
| TC20 | Security     | Unauthenticated access        | Redirected to login page                          | Access denied           | Pass   |
| TC21 | Location     | GPS coordinate capture        | Latitude/longitude stored with listing            | Coordinates saved       | Pass   |
| TC22 | Location     | Reverse geocoding             | Address string returned from coordinates          | Address displayed       | Pass   |
| TC23 | Notification | Status change notification    | Notification created in Firestore                 | Notification sent       | Pass   |
| TC24 | Notification | Unread count badge            | Badge shows correct unread count                  | Count shown correctly   | Pass   |
| TC25 | Farmer       | Export CSV                    | CSV file downloaded with listing data             | CSV exported            | Pass   |
| TC26 | Farmer       | View carbon credits           | Carbon panel shows offset values                  | Credits displayed       | Pass   |
| TC27 | Agent        | Claim already-claimed listing | Error message shown                               | Error message displayed | Pass   |
| TC28 | Admin        | View platform statistics      | User and listing counts displayed                 | Statistics shown        | Pass   |
| TC29 | Responsive   | Mobile viewport rendering     | All elements visible and usable                   | Layout responsive       | Pass   |
| TC30 | Performance  | Dashboard load time           | Page loads within 3 seconds                       | Loaded in 1.8s          | Pass   |

Table 10.1: Test Cases and Results

### 10.3 Performance Benchmarks

The following table presents performance benchmarks measured during testing:

Table 10.2: Performance Benchmarks

| Metric                      | Target       | Measured          | Status |
|-----------------------------|--------------|-------------------|--------|
| Initial Page Load (Landing) | < 3 seconds  | 1.4 seconds       | Pass   |
| Dashboard Load (Farmer)     | < 3 seconds  | 1.8 seconds       | Pass   |
| Dashboard Load (Admin)      | < 3 seconds  | 2.1 seconds       | Pass   |
| Firestore Query Latency     | < 500ms      | 180ms (avg)       | Pass   |
| Image Upload (Cloudinary)   | < 5 seconds  | 2.3 seconds (avg) | Pass   |
| Razorpay Checkout Modal     | < 2 seconds  | 1.1 seconds       | Pass   |
| Carbon Credit Calculation   | < 100ms      | 5ms               | Pass   |
| TypeScript Compilation      | < 30 seconds | 12 seconds        | Pass   |

## 10.4 Testing Results Summary

All 30 test cases passed successfully. Key findings from the testing process:

- **Zero Critical Bugs:** No critical or blocking issues were found during the final testing phase.
- **Type Safety:** TypeScript's strict mode caught 23 potential runtime errors during compilation that would have been undetectable in plain JavaScript.
- **Security Compliance:** All Firestore security rules properly enforce role-based access. No unauthorized data access was possible during penetration testing.
- **Performance:** All pages load within the 3-second target on standard broadband connections. Firestore query performance averages 180ms across all tested operations.
- **Cross-Browser:** The application renders consistently across all tested browsers with no layout breaking or functionality loss.